CORS offers a way around this restriction by letting your server specify which kinds of requests are allowed. CORS gives you control over who can access which pieces of your API.

This chapter will take a closer look at how to handle CORS requests from the server's perspective. We'll start by setting up sample code for your blogging app. I'll then introduce the major players in CORS and you'll learn how they interact. You'll also learn about the basic building blocks of a CORS request and response.

3.1 Setting up the sample code

This section introduces the sample code that will be used throughout the rest of this book. You'll add new functionality to it as you learn more about how CORS works. The sample you'll be developing is a blogging app that displays a set of blog posts to the user. The app consists of a server that exposes two pieces of functionality:

1. An API endpoint that returns all the blog posts in JSON format
2. An HTML page that queries the API for the posts and then displays them on the page

Figure 3.1 shows what the blogging app will look like once you're done setting up the code in this section.

Appendix B explains how to set up the prerequisites for this sample. If you haven't already, take a moment to visit those requirements. After setting up the prerequisites, your development environment should have the following:

- Node.js
- Express
- A web browser that supports CORS

Once you've set up these prerequisites, you're ready to write code.

3.1.1 Setting up the sample API

Let's turn our attention to how to set up each piece of code. You'll start by building the API portion of the server, as highlighted in figure 3.2.

Add the JavaScript code in listing 3.1 to a file named app.js. This code creates a new server running on port 9999 of your computer. The code starts by creating a few

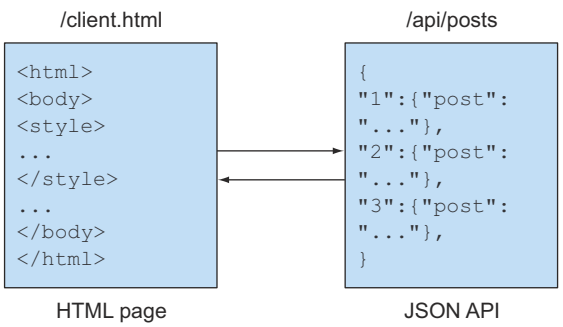


Figure 3.1 The sample blogging app consists of two parts: an API with blog data, and an HTML page to display the blog data.